COMPTE RENDU

SERRURE ELECTRONIQUE - VHDL
DESCRIPTION - SIMULATION - SCHEMAS

1 - Description du projet
2 - Description des outils utilisés
3 - Projet
4 - Synthèse et conclusion

1. Description du projet

L'objectif est de concevoir et simuler une serrure électronique sur carte DE10-Lite. Le système permet la saisie d'un code à 4 chiffres via des interrupteurs, la validation par bouton-poussoir (KEY0) et l'affichage sur 7 segments. La serrure se déverrouille (signal `ouverture` à '1') uniquement si le code saisi correspond au code de référence (ex: `1234`).

2. Outils pour le projet

- Vscode pour rédiger les code vhdl des entités vhdl et leurs architectures. Ainsi que leurs test bench correspondant.
- Logisim Evolution pour le/les schéma(s) de la serrure et de certains composants
- Modelsim pour la simulation et la vérification du bon-fonctionnement des composants et de la serrure.

3. Projet

Les notations 4.X correspondent à celles de l'énnoncé pour plus de visibilité

4.1 Detecteur de front

Le detecteur de front nous permet de savoir si KEY0 est préssé

src > detect.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.all;



entity detect is
port(
    clk : in STD_LOGIC;
    rst : in std_logic;
    KEY0 : in std_logic;
    enable : out std_logic);
end entity detect;

architecture behave of detect is
signal KEY0_transition : std_logic := '0';
begin

    process(KEY0, clk, rst)
    begin

        if rst = '0' then
            enable <= '0';
            KEY0_transition <= '0';
        elsif RISING_EDGE(clk) then
            KEY0_transition <= KEY0;
            enable <=  (not KEY0_transition) and KEY0;
        end if;

    end process;
end architecture;
```

On notera qu'une source d'erreur a été de mettre à 1 le rst alors que c'est un reset bas pour tous les autres composants du circuit

4.2 Bascule D active sur front Montant

src > basculed.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.all;


entity bascule is
    port ( clk     : in  std_logic;
           rst     : in  std_logic;
           D       : in  std_logic;
           Q       : out std_logic);
end entity bascule;

architecture yo of bascule is
begin
    process(clk, rst)
    begin
        if rst = '1' then
            Q <= '0';
        elsif rising_edge(clk) then
            Q <= D;
        end if;
    end process;
end architecture;
```

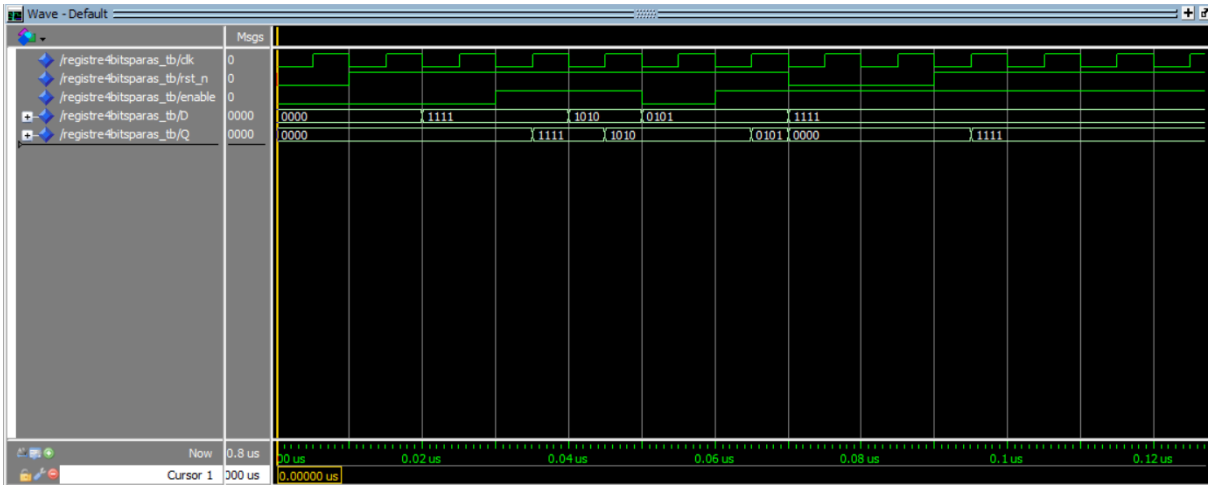
4.3 Registre à chargement parallèle (4 bits)

Code :

src > registre4bitsparas.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity registre4bits is
    port (
        clk : in std_logic;
        rst_n : in std_logic;
        enable : in std_logic;
        D : in std_logic_vector(3 downto 0);
        Q : out std_logic_vector(3 downto 0)
    );
end entity registre4bits;

architecture REG of registre4bits is
begin
    process(clk, rst_n)
    begin
    if rst_n = '0' then
        Q <= "0000";
    elsif rising_edge(clk) then
        if enable = '1' then
            Q <= D;
        end if;
    end if;
    end process;
end architecture REG;
```

Simulation :



Q charge bien la valeur de D sur 4 bits lorsque enable vaut 1 et conserve cette valeur lorsque enable vaut 0. De plus le reset bas fonctionne on peut le constater à 0.06us

On observe bien le maintien de la valeur quand `enable` est à '0' et la remise à zéro via `rst_n`.

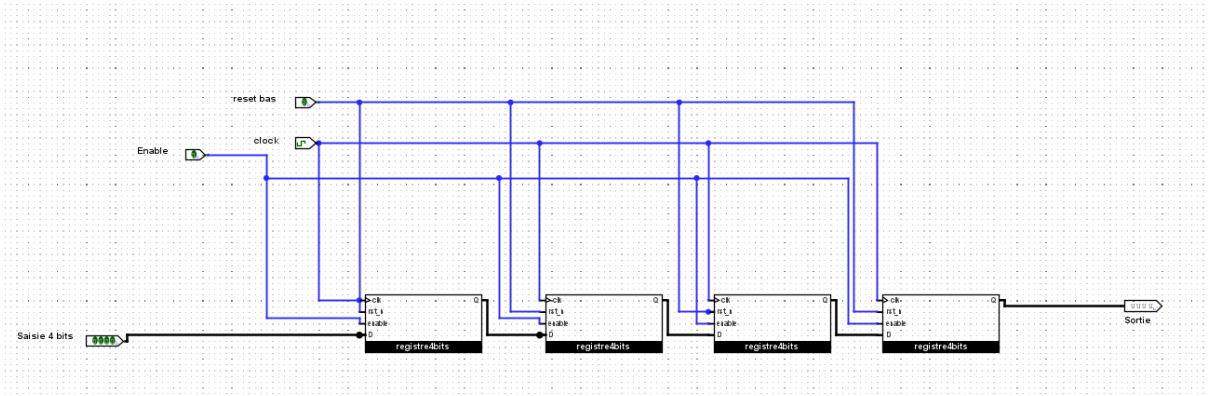
4.4 - Module de Stockage du Code :

4 registres 4 bits en série

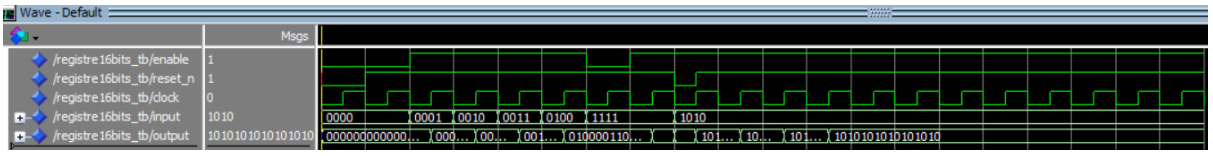
Code :

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity registre4bits is
    port (
        clk : in std_logic;
        rst_n : in std_logic;
        enable : in std_logic;
        D : in std_logic_vector(3 downto 0);
        Q : out std_logic_vector(3 downto 0)
    );
end entity registre4bits;

architecture REG of registre4bits is
begin
    process(clk, rst_n)
    begin
    if rst_n = '0' then
        Q <= "0000";
    elsif rising_edge(clk) then
        if enable = '1' then
            Q <= D;
        end if;
    end if;
    end process;
end architecture REG;
```

Schéma :



Simulation :

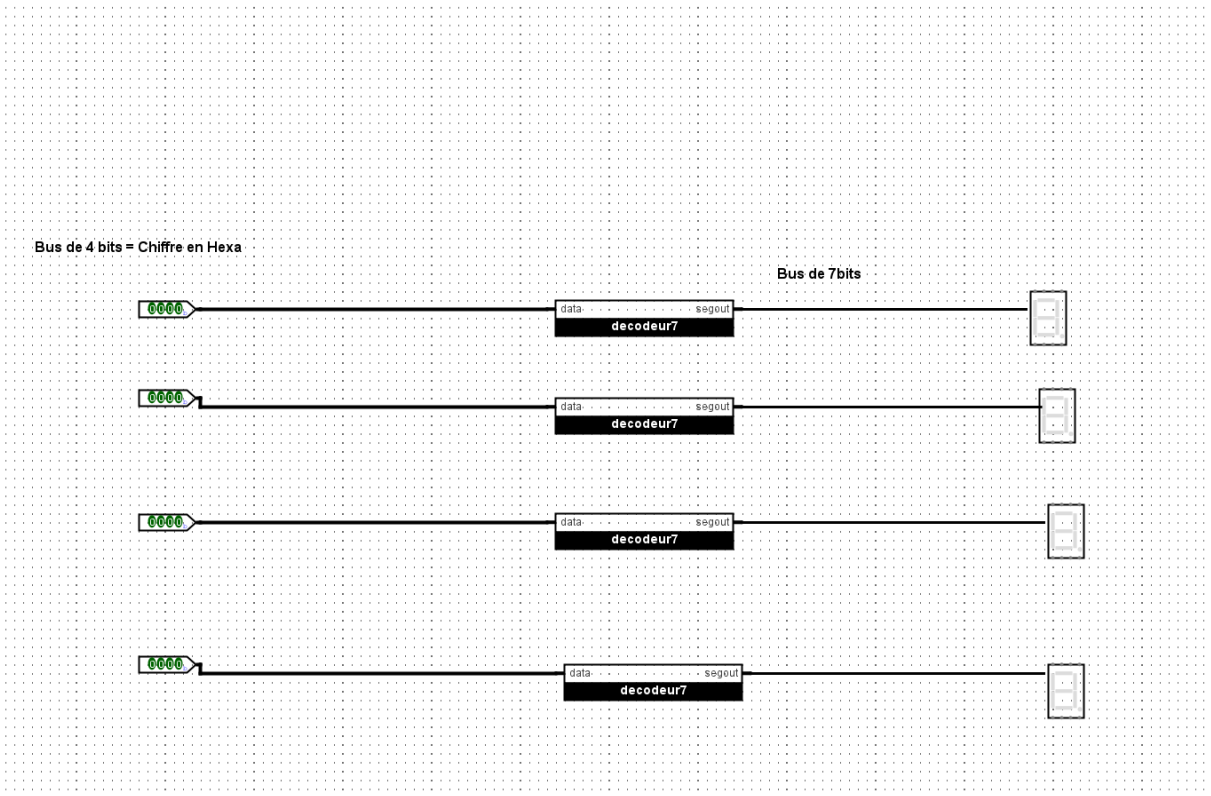


Vous trouverez sur également le code du fichier .do :

taper la commande : *do simu/simuRegistre16bits.do*

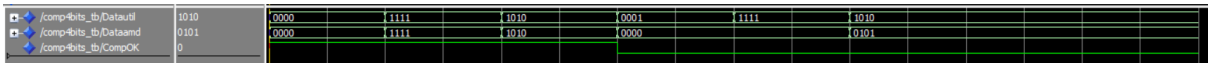
puis “Zoom Full” ou “F”.

4.5 - Afficheur 7 SEGEMENTS



4.6 - Comparateur 4 bits :

Le comparateur n'a pas besoin de reset pour mettre sa sortie à 0 car c'est un élément purement combinatoire du circuit. Equivalent à une porte ET.



Vous trouverez également le code du fichier .do :

taper la commande : *do simu/simucomp4bits.do*

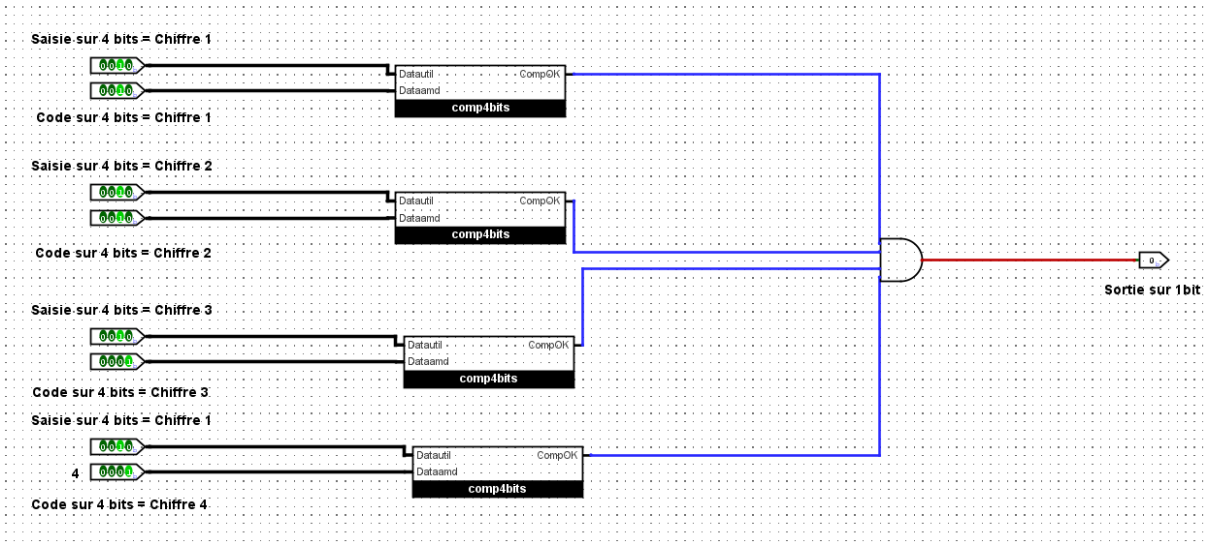
4.7 : Comparateur 16 bits

Code :

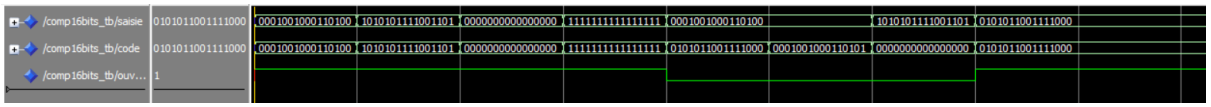
src > comp16bits.vhd

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.all;


entity comp16bits is

    port(
        saisie : in STD_LOGIC_VECTOR(15 downto 0);
        code : in STD_LOGIC_VECTOR (15 downto 0);
        ouverture : out std_logic
    );

end entity comp16bits;


architecture ARCH of comp16bits is
    signal output : STD_LOGIC_VECTOR(3 downto 0);
begin

    comparateur1 : entity work.comp4bits port map(Datautil => saisie(3 downto 0), Dataamd => code(3 downto 0), CompOK => output(0));
    comparateur2 : entity work.comp4bits port map(Datautil => saisie(7 downto 4), Dataamd => code(7 downto 4), CompOK => output(1));
    comparateur3 : entity work.comp4bits port map(Datautil => saisie(11 downto 8), Dataamd => code(11 downto 8), CompOK => output(2));
    comparateur4 : entity work.comp4bits port map(Datautil => saisie(15 downto 12), Dataamd => code(15 downto 12), CompOK => output(3))

    ouverture <= '1' when output = "1111" else '0';
end architecture;
```

Schéma : (On a ajouter une porte logique AND pour comparer les sorties de de chaque comparateur 4bits)



Simulation :



Script de simulation : simu/simucomp16bits

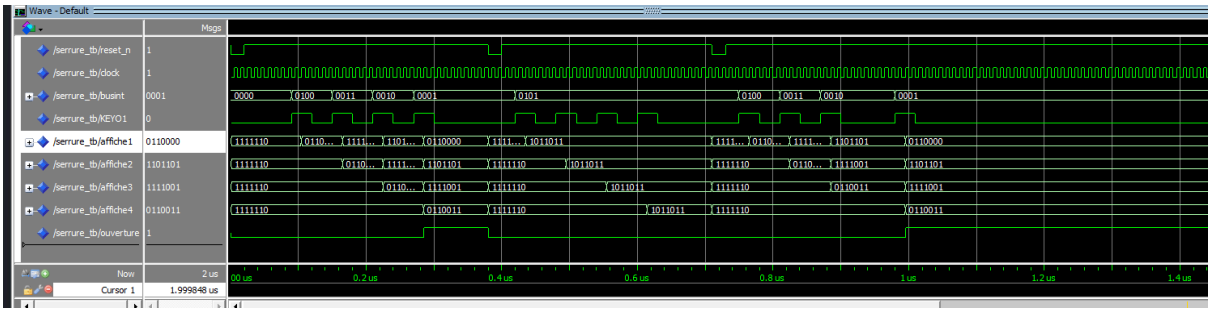
```
simu > simucomp16bits.do
vlib work

#Compilation des fichiers src

vcom -93 src/comp4bits.vhd
vcom -93 src/comp16bits.vhd


#compile du test_bench

vcom -93 simu/comp4bits_tb.vhd
vcom -93 simu/comp16bits_tb.vhd

vsim work.comp16bits_tb

add wave *

run 40ns
wave zoom full
```

4.8 Serrure

Code :

Vous trouverez le code vhd de la serrure dans src/serrure.vhd
Le testbench correspondant dans testsbenchs/serrure_tb.vhd
Et le script de simulation de la serrure complète dans simu/simu.do

Simulation :



On constate bien que le signal ouverture passe à l'état haut lorsque le bon code est entrée.

4. Conclusion

Le projet a permis de valider la chaîne complète de conception VHDL, de la description structurelle à la simulation temporelle. Les principales difficultés rencontrées résidaient dans la gestion de la polarité des signaux de reset et l'ordre de stockage des données dans la mémoire à décalage, problèmes résolus par l'analyse des chronogrammes sous ModelSim.